

ASSESSMENT - PSEUDO CODE WRITING.
Q1: DESIGNING CLASS HIERARCHY (PSEUDO CODE)

CLASS Person

DATA

name

age

METHOD Person(name, age)

 this.name ← name

 this.age ← age

END METHOD

END CLASS

CLASS Student Inherits person

DATA

registerNumber

marks[5]

METHOD Student(name, age, registerNumber,
marks[5])

 SUPER(name, age)

 this.registerNumber ← registerNumber

 this.marks ← marks

END METHOD

METHOD calculateAverage()

 total ← 0

 FOR i ← 0 TO 4

 total ← total + marks[i]

 END FOR

 RETURN total/5

END METHOD

END CLASS

```
CLASS ResearchStudent INHERITS Student
```

```
DATA
```

```
    researchArea.
```

```
METHOD ResearchStudent(name, age, registerNumber,  
                           marks[5], researchArea)
```

```
    SUPER(name, age, registerNumber, marks)
```

```
    this.researchArea ← researchArea
```

```
END METHOD
```

```
METHOD displayCompleteDetails()
```

```
    PRINT "Name: " + name
```

```
    PRINT "Age: " + age
```

```
    PRINT "Register Number: " + registerNumber
```

```
    PRINT "Average Marks: " + calculateAverage()
```

```
    PRINT "Research Area: " + researchArea
```

```
END METHOD
```

```
END CLASS
```

```
BEGIN
```

```
    r.s ← NEW ResearchStudent("Anitha", 24, "R1023",  
                               [88, 91, 79, 85, 93], "machine learning")
```

```
    r.s displayCompleteDetails()
```

```
END
```

Q2 : COMPLETE THE MISSING PSEUDO CODE :
BEGIN

 READ N

 READ ARRAY A

 FOR $i \leftarrow 0$ TO $N-1$

 FOR $j \leftarrow i+1$ TO $N-1$

 IF $A[i] == A[j]$

$A[j] \leftarrow -1$

 ENDIF

 ENDFOR

 ENDFOR

FOR $i \leftarrow 0$ TO $N-1$

IF $A[i] \neq -1$

PRINT $A[i]$

END IF

END FOR

END

Q3 : DEBUG THE PSEUDO CODE

CLASS AREA

METHOD area(length)

RETURN length * length

END METHOD

METHOD area(length, breadth)

RETURN length * breadth

END METHOD

END CLASS

Q4: Trace the execution:

BEGIN

SUM $\leftarrow$ 0

FOR $i \leftarrow 1$ TO 5

IF $i \bmod 2 = 0$

SUM $\leftarrow$ SUM + i

ELSE

SUM $\leftarrow$ SUM - i

ENDIF

END FOR

PRINT SUM

END

output:
-3

Q5:

BEGIN

READ N

READ ARRAY A

first $\leftarrow$ -INFINITY

Second $\leftarrow$ -INFINITY

FOR $i \leftarrow 0$ TO $N-1$

IF $A[i] > \text{first}$

Second $\leftarrow$ first

first $\leftarrow A[i]$

ELSE IF $A[i] > \text{second}$ AND $A[i] \neq \text{first}$

Second $\leftarrow A[i]$

END IF

END FOR

IF Second == -INFINITY

PRINT "No second largest distinct element
exist"

ELSE

PRINT second

END IF

END

Q6: Convert problem statement to Pseudo code:

```
INTERFACE PaymentMethod.
```

```
    METHOD pay(amount)
```

```
END INTERFACE
```

```
CLASS CreditCard IMPLEMENTS PaymentMethod
```

```
    METHOD pay(amount)
```

```
        PRINT "PAID" + amount + "using CreditCard"
```

```
    END METHOD.
```

```
END CLASS
```

```
CLASS UPI IMPLEMENTS PaymentMethod
METHOD pay(amt)
    PRINT "paid" + amt +
        "using UPI"
END METHOD
END CLASS
```

```
CLASS NetBanking IMPLEMENTS PaymentMethod
METHOD pay(amt)
    PRINT "paid" + amt +
        "using Netbanking"
END METHOD
```

```
END CLASS
```

```
( BEGIN
```

```
    READ choice
```

```
    READ amount
```

```
    PaymentMethod method
```

```
    IF choice == "CreditCard"
```

```
        method ← new CreditCard()
```

```
    ELSE IF choice == "UPI"
```

```
        method ← new UPI()
```

```
    ELSE IF choice == "Net Banking"
```

```
        method ← new NetBanking()
```

```
    END IF
```

```
    method.pay(amount)
```

```
END.
```

Q7: Runtime Polymorphism:

```
CLASS Animal
```

```
    METHOD speak()
```

```
        PRINT "Animal"
```

```
    END METHOD
```

```
END CLASS
```

```
CLASS Dog INHERITS Animal
```

```
    METHOD speak()
```

```
        PRINT "Barks"
```

```
    END METHOD
```

```
END CLASS
```

```
Animal obj ← New Dog()
```

```
obj.speak()
```

Output: Barks

Q8: Find logical errors (constructors)

```
CLASS Employee
```

```
    DATA
```

```
        id
```

```
        name
```

```
    CONSTRUCTOR (empId, empName)
```

```
        id ← empId
```

```
        name ← empName
```

```
    END CONSTRUCTOR
```

```
END CLASS
```

```
BEGIN
```

```
    e ← NEW Employee(101, "Ravi Kumar")
```

```
END
```

Q9 → case study: vehicle management

CLASS Vehicle

DATA

VehicleNO

OwnerName

METHOD Vehicle(VehicleNO, OwnerName)

this.VehicleNO ← VehicleNO

this.OwnerName ← OwnerName

END METHOD

METHOD display()

PRINT "vehicle No: " + VehicleNO

PRINT "Owner Name: " + OwnerName

END METHOD

END CLASS

CLASS Car INHERITS Vehicle

DATA

FuelType

METHOD Car(VehicleNO, OwnerName, FuelType)

SUPER(VehicleNO, OwnerName)

this.FuelType ← FuelType

END METHOD

END CLASS

CLASS Truck INHERITS Vehicle

DATA

loadCapacity

METHOD Truck(VehicleNO, OwnerName,

loadCapacity)

SUPER(VehicleNO, OwnerName)

this.loadCapacity ← loadCapacity

END METHOD

```
METHOD display()
```

```
    SUPER.display()
```

```
    PRINT "Load Capacity: " + loadCapacity.
```

```
END METHOD
```

```
END CLASS
```

```
BEGIN
```

```
    READ N
```

```
    vehicle Vehicles[N]
```

```
    FOR i ← 0 TO N-1
```

```
        READ type
```

```
        READ vehicleNo, ownerName
```

```
        IF type == "Car"
```

```
            READ fuelType
```

```
            Vehicles[i] ← New Car(vehicleNo,  
                                   ownerName, fuelType)
```

```
        ELSE IF type == "Truck"
```

```
            READ loadCapacity
```

```
            Vehicles[i] ← New Truck(vehicleNo,  
                                     ownerName, loadCapacity)
```

```
        ENDIF
```

```
    END FOR
```

```
    FOR i ← 0 TO N-1
```

```
        Vehicles[i].display()
```

```
    ENDFOR
```

```
END
```